

Batch analysis of cosmic rays using Drift Tubes detectors

Management and Analysis of Physics Dataset (Mod. B)



UNIVERSITÀ
DEGLI STUDI
DI PADOVA

Group 12:
Cafagno Samuele
Cavallin Jonathan
Di Lucia Sofia

Project Introduction

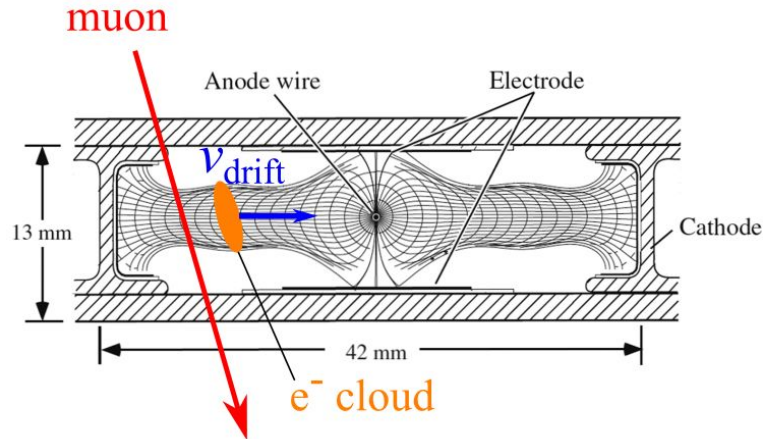


UNIVERSITÀ
DEGLI STUDI
DI PADOVA

Goal of the Project

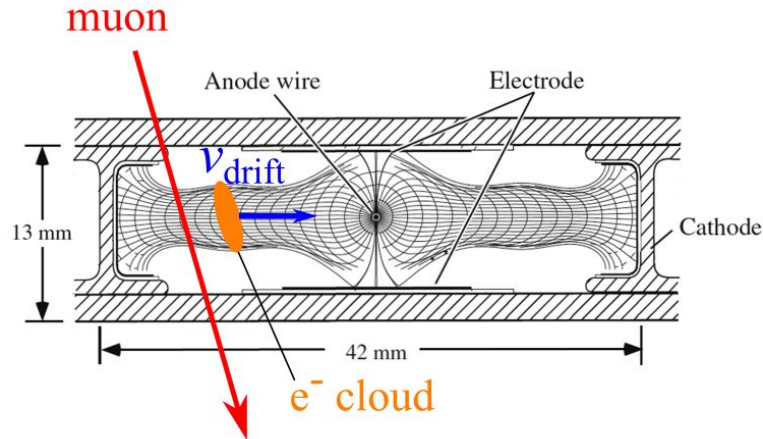
The project aims to analyze real data from a particle physics detector to reconstruct the trajectories of muons produced by cosmic rays. To achieve this, we employed **distributed data processing techniques**.

The dataset was collected from a set of muon detectors, called **mini Drift Tubes** (miniDTs), installed at the Legnaro INFN Laboratories.



Mini-DTS

- **Operate by Ionization:** the passage of the muons inside the chambers ionize the gas, creating electron-ion pairs.
- The **electrons drift** toward the anode wire at a constant drift **velocity** $V_{drift} = 53.8 \mu\text{m/ns}$ due to the electric field.
- The signals are amplified and digitized by the DAQ (Data Acquisition System).
- For each ionization event (**hit**) the FPGAs perform the Time-to-Digital Conversion (TDC).





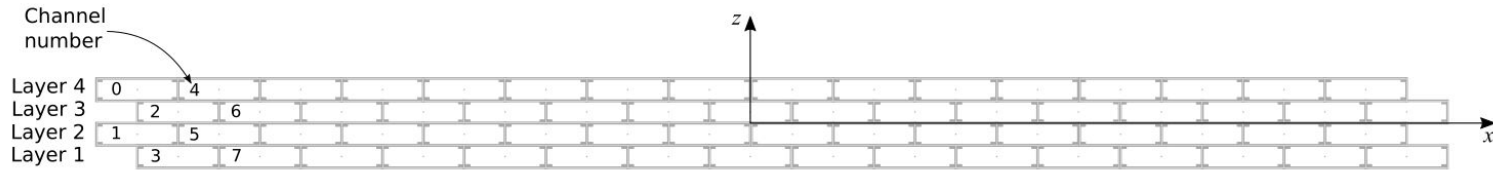
- The chambers record the time information t_{hit} of the **signal arrival to the anode**. To measure the **actual time of passage** of the muon t_0 , two external plastic scintillator are used.
- Knowing the time information of each hit, the drift velocity and the geometry of the detector, it's possible to obtain the distance from the anode at which the muon passed

$$x_{hit} = (t_{hit} - t_0)v_{drift}$$

- This way, each hit position x_{hit} in a cell has an intrinsic **left/right ambiguity** with respect to the anode; to solve it we need to gather hit data from more than one cell.

Full Detector Geometry and Structure

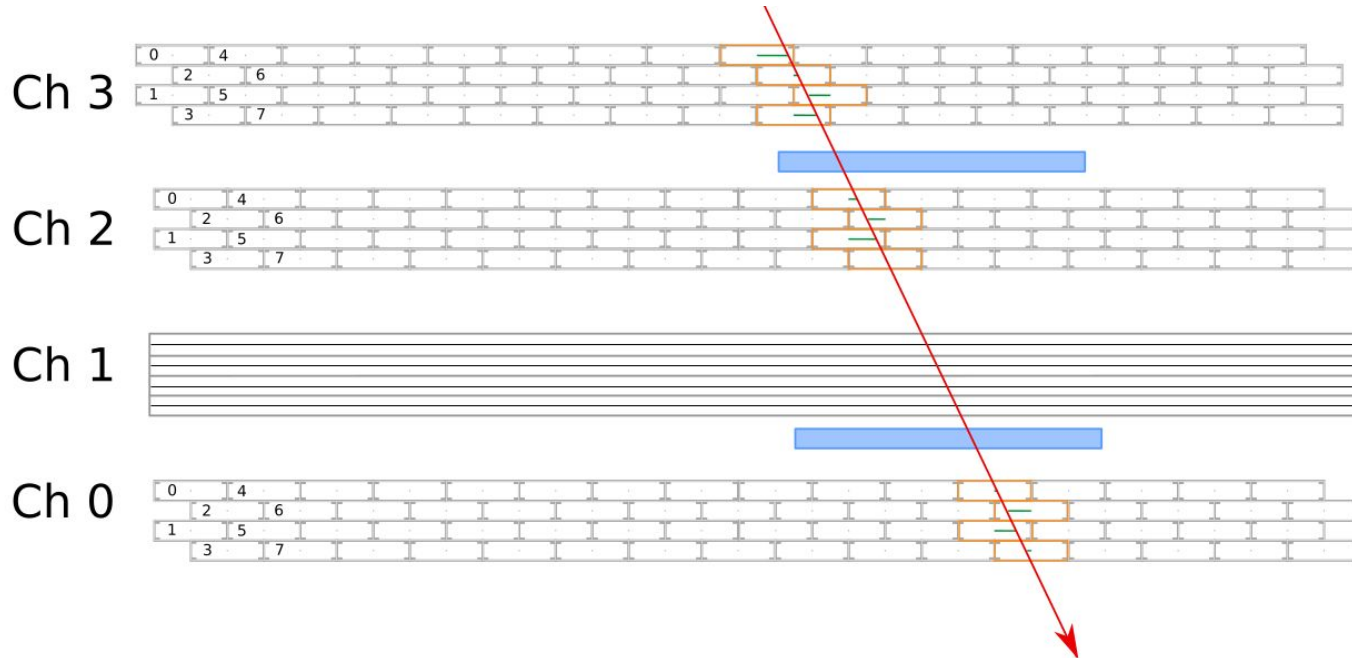
- Each drift tube **cell** is $42 \times 13 \text{ mm}^2$ (width $\times$ height), with a length of approximately 60cm
- Each **chamber** is made of 64 cells divided in 4 layers of 16 cells each, with each layer is shifted by half a cell with respect to the one above. To each cell a **channel number** is assigned according to a recurrent pattern:



- The full detector is made of 4 stacked chambers, with one of them rotated by 90 degrees to provide a measure from an orthogonal view.



Full Detector Schema



Chambers 3,2 and 0 provide data on the same plane, while chamber 1 provides the orthogonal data. The blue rectangles represent the plastic scintillator palettes

Setup of the cluster



UNIVERSITÀ
DEGLI STUDI
DI PADOVA

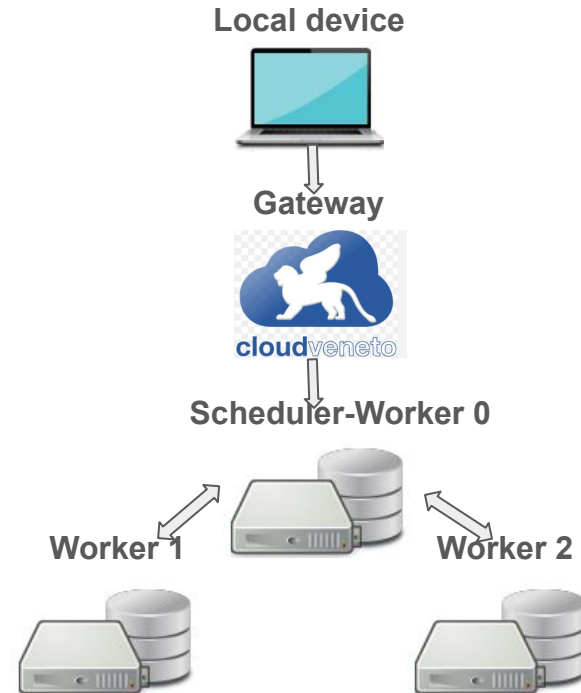
Cluster Setup

For this project we set up a cluster of Virtual Machines hosted on the Cloud Veneto infrastructure:

Virtual Machines' Specifics:

- **Scheduler + Worker 0 (10.67.22.48):**
RAM 16GB, 8 VCPUs, Disk 25GB
- **Worker 1 (10.67.22.229):** RAM 8GB, 4 VCPUs, Disk 25GB
- **Worker 2 (10.67.22.45):** RAM 8GB, 4 VCPUs, Disk 25GB

We chose to make use of the **Dask Distributed Framework**.





Cluster Setup

- We use **SSH protocols** to access the VMs of the cluster.
- We set up a passwordless SSH connection between the VMs, thus we only need to connect to the Scheduler to access the whole cluster.
- Then we set up the Dask cluster specifying the Scheduler's and workers' IPs, the **number of workers per VM** and the **number of threads per worker**:

```
cluster = SSHCluster(  
    ["10.67.22.48", "10.67.22.48", "10.67.22.229", "10.67.22.45"],  
    worker_options={"nthreads": nthreads, "n_workers": n_workers},  
    scheduler_options={"port": 0, "dashboard_address": ":40973"},  
    connect_options={  
        "known_hosts": None,  
        "client_keys": "cluster_key"})  
  
client = Client(cluster)
```



Cluster Setup



Client

Client-eca91ee5-8745-11f0-aa4e-c70832010a25

Connection method: Cluster object

Cluster type: distributed.SpecCluster

Dashboard: <http://10.67.22.48:34395/status>

[Launch dashboard in JupyterLab](#)

▼ Cluster Info



SpecCluster

SSHCluster

Dashboard: <http://10.67.22.48:34395/status>

Workers: 3

Total threads: 3

Total memory: 5.83 GiB

▼ Scheduler Info



Scheduler

Scheduler-2d40570b-ee4b-4012-b7af-8d6ba77ba5f3

Comm: <tcp://10.67.22.48:38535>

Workers: 0

Dashboard: <http://10.67.22.48:34395/status>

Total threads: 0

Started: Just now

Total memory: 0 B

▼ Workers



▼ Worker: <tcp://10.67.22.229:42265>

Comm: <tcp://10.67.22.229:42265>

Total threads: 1

Dashboard: <http://10.67.22.229:37263/status>

Memory: 1.94 GiB

Nanny: <tcp://10.67.22.229:41273>

Local directory: /tmp/dask-scratch-space/worker_qipyd2p



▼ Worker: <tcp://10.67.22.45:42431>

Comm: <tcp://10.67.22.45:42431>

Total threads: 1

Dashboard: <http://10.67.22.45:36967/status>

Memory: 1.94 GiB

Nanny: <tcp://10.67.22.45:37131>

Local directory: /tmp/dask-scratch-space/worker-9sxewzye



▼ Worker: <tcp://10.67.22.48:34207>

Comm: <tcp://10.67.22.48:34207>

Total threads: 1

Dashboard: <http://10.67.22.48:45651/status>

Memory: 1.95 GiB

Pre-processing



UNIVERSITÀ
DEGLI STUDI
DI PADOVA



Pre-processing

- First, we need to read raw data, which is provided on a cloud storage s3 bucket hosted on Cloud Veneto. Every hit is encoded as a **64-bit binary word**.
- We give a structure to our data defining a **Pandas Dataframe**, extracting each different bit group with **bit-masking** operations.
- We are only interested in TDC hits data tagged by a header **HEAD=2** (muons), thus we filter out the rest.

```
def read_file(file):
    client_kwargs={
        "endpoint_url": "https://cloud-areapd.pd.infn.it:5210", "verify": False}
    # block is a list of Dask delayed objects, representing chunks of bytes from a file
    small, blocks = dask.bytes.read_bytes(
        file,
        key="32b22133696b4aad9a2872435d392277",
        secret="d2111717321d4f7dbd6755b98e95e713",
        client_kwargs=client_kwargs)
    # apply converter to each block lazily (each block object is a tuple, so i select [0])
    dfs = [dask.delayed(bites_converter)(block[0]) for block in blocks]
    # converts a list of Dask delayed objects (chunks of a DataFrame) into a unique Dask DataFrame
    ddf = dd.from_delayed(dfs)
    return(ddf)

@dask.delayed
def bites_converter(block):
    # organize in numpy arrays of 64 bits
    byte_array = np.frombuffer(block, dtype=np.uint64)

    # bit masking
    head = (byte_array >> 61) & 0x7          # select bits 61-63 (3 bits)
    fpga = (byte_array >> 58) & 0x7          # select bits 58-60 (3 bits)
    chan = (byte_array >> 49) & 0x1FF        # select bits 49-57 (9 bits)
    orbit = (byte_array >> 17) & 0xFFFFFFF  # select bits 17-48 (32 bits)
    bx = (byte_array >> 5) & 0xFFFF         # select bits 5-16 (12 bits)
    tdc = (byte_array >> 0) & 0x1F          # select bits 0-4 (5 bits)

    # we are only interested in data with head==2
    df = pd.DataFrame(
        {"FPGA": fpga[head==2],
         "CHAN": chan[head==2],
         "ORBIT": orbit[head==2],
         "BX": bx[head==2],
         "TDC": tdc[head==2]} )

    return(df)
```

Pre-processing

- We **assign** the **chamber number** to each event based on the FPGA and CHANNEL values.
- We isolate the information regarding the scintillators to define the passage times t_0 in ns using the conversion:
$$t_0 = 25ns \left(ORBIT \times 3564 + BX + \frac{TDC}{30} \right)$$
- We filter events in the dataframe considering **only ORBITs with a single scintillator hit** recorded.

```
# assign chamber number
```

```
df = df.assign(CHAMBER = -1)
df["CHAMBER"] = df["CHAMBER"].mask((df["FPGA"] == 0) & (df["CHAN"] <= 63), 0)
df["CHAMBER"] = df["CHAMBER"].mask((df["FPGA"] == 0) & (df["CHAN"] > 63), 1)
df["CHAMBER"] = df["CHAMBER"].mask((df["FPGA"] == 1) & (df["CHAN"] <= 63), 2)
df["CHAMBER"] = df["CHAMBER"].mask((df["FPGA"] == 1) & (df["CHAN"] > 63), 3)
```

```
# select scintillator hits
```

```
scintillator = df[(df["FPGA"]==1) & (df["CHAN"]==128)].copy()
scintillator = scintillator[["ORBIT", "BX", "TDC"]]
scintillator["t_0"] = 25*(scintillator["ORBIT"]*3564 + scintillator["BX"] + scintillator["TDC"]/30)
```

```
# count scintillator hits per ORBIT
```

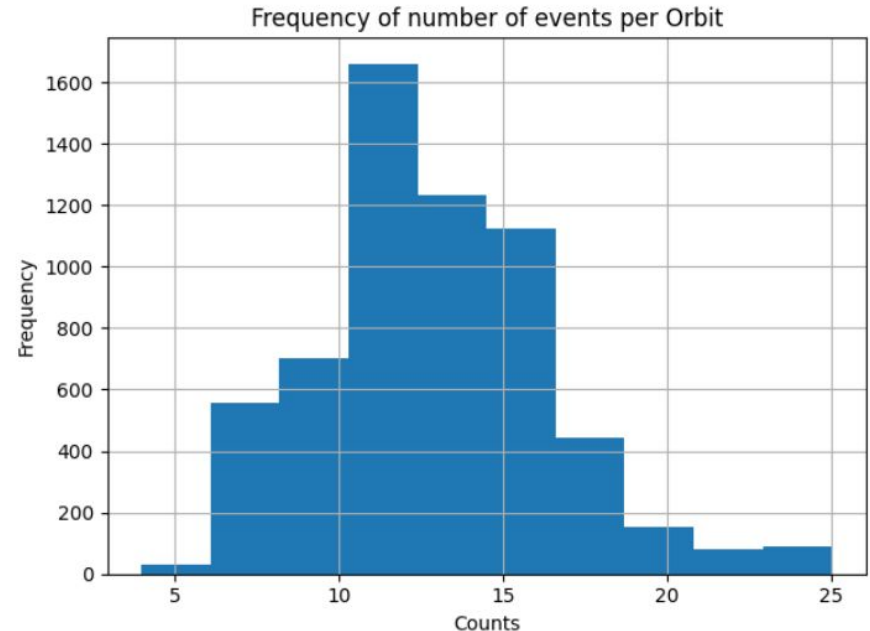
```
scint_counts = scintillator.groupby("ORBIT").size().to_frame("scint_hits").reset_index()
# keep only ORBITs with a single scintillator hit
valid_orbits = scint_counts[scint_counts["scint_hits"] == 1]["ORBIT"]
# filter scintillator hits and main df: keep only events with 1 scintillator hit per ORBIT
scintillator = scintillator[scintillator["ORBIT"].isin(valid_orbits)]
df = df[df["ORBIT"].isin(valid_orbits)]
```



Pre-processing

```
# consider only events with ≤ num_hits chamber hits per ORBIT (avoid overcrowded events)
chamber_counts = df.groupby("ORBIT").size().to_frame("hits_count").reset_index()
valid_orbits = chamber_counts[(chamber_counts["hits_count"] ≤ num_hits) & (chamber_counts["hits_count"] ≥ 4)][["ORBIT"]]
df = df[df["ORBIT"].isin(valid_orbits["ORBIT"])]
```

- We further filter out from the dataset ORBITs with less than 4 and more than ***num_hits*** total chamber hits, in order to be able to reconstruct at least a full trajectory in a chamber and also avoid some likely external noise. We chose ***num_hits=25*** in our analysis (cautious not to discard too many).





Pre-processing

- We define a **time offset correction** to the scintillator times, to account for the **delay of the signal** at different distances between each chamber and the relative scintillator.
- We convert in times (ns) the information from the chamber hits defining t_{hit} similarly to what we did for t_0 .
- We can now **compute the relative times t_{rel}** between the muon passages and the signals at the anodes (electrons drift time).

```
# scintillator time offset correction, per chamber
df = df.assign(time_offset = 0.0)
df["time_offset"] = df["time_offset"].mask(df["CHAMBER"] == 0, 95.0-1.1)
df["time_offset"] = df["time_offset"].mask(df["CHAMBER"] == 1, 95.0+6.4)
df["time_offset"] = df["time_offset"].mask(df["CHAMBER"] == 2, 95.0+0.5)
df["time_offset"] = df["time_offset"].mask(df["CHAMBER"] == 3, 95.0-2.6)

df["T_HIT"] = 25*(df["ORBIT"]*3564 + df["BX"] + df["TDC"])/30)

# calculate relative chamber hit times
df["T_REL"] = df["T_HIT"] - (df["t_0"]-df["time_offset"])
```




Pre-processing

- We **define a reference system** centered horizontally at the midpoint of the chambers and vertically originating from the ground.
- We compute the drift distance of the electrons from the relative times and thus **define the (x,z) coordinates** of each chamber hits, with the ambiguity for the horizontal coordinates between x_{left} and x_{right}
- We **persist** the obtained processed Pandas **Dataframe**.

```
# compute x and z coordinates of the chamber hits (left and right ambiguity for the x coordinate in each hit)
def compute_coordinates(df):

    df["x"]=-7.5*42.0 # center of the first cell to the left in each chamber
    df["z"]=0

    # assign the z-position of the center of each chamber w.r.t. to the floor
    df["z"]=df["z"].mask(df["CHAMBER"]==0,219.8)
    df["z"]=df["z"].mask(df["CHAMBER"]==1,977.3)
    df["z"]=df["z"].mask(df["CHAMBER"]==2,1035.6)
    df["z"]=df["z"].mask(df["CHAMBER"]==3,1819.8)

    df["CHAN_LOCAL"] = df["CHAN"]
    df["CHAN_LOCAL"] = df["CHAN_LOCAL"].mask(df["CHAMBER"].isin([1,3]), df["CHAN"]- 64)

    layer_map = [3, 1, 2, 0]
    df["layer"] = df["CHAN_LOCAL"]%4
    df["layer"] = df["layer"].map({i: layer_map[i] for i in range(4)})

    df["x"]= df["x"] + df["CHAN_LOCAL"]// 2 * 21.
    df["z"] = df["z"] - 26.0 + (df["layer"]+0.5)*13.

    df["drift"]= df["T_REL"]*53.8*0.001 # v_drift = 53.8um/ns
    df = df[(df["drift"]>=0)&(df["drift"]<=21.)] # drift can be at most half of the cell, discard other events (meaningless)

    df["X_RIGHT"]= df["x"]+df["drift"]
    df["X_LEFT"]= df["x"]-df["drift"]

    return(df)

df = read_file("s3://project-bkp/data_00000*") # use all files
df = df.map_partitions(first_processing,num_hits=25)
new_df = df.map_partitions(compute_coordinates)
persisted = new_df.persist()
```

Local track Fit



UNIVERSITÀ
DEGLI STUDI
DI PADOVA



Local track fit

- We perform a linear fit over all the possible combination of $\mathbf{x}_{hit}$ for each chamber separately.
- Extracting the **angular parameter** and **intercept** will give us the local track (limited to a single chamber) of the muon considered.
- This **solves the left/right ambiguity** as we pick only one out of the many possible combinations of points as the real track points.
- The fit selects as the best the combinations of $\mathbf{x}_{hit}$ with the lowest squared residuals normalized by the number of degree of freedom (ndof).

$$\chi^2/ndof = \frac{\sum (y_{obs} - y_{fit})^2}{ndof}$$



Local fit algorithm

We implemented a **vectorized algorithm**, avoiding using explicit for loops over every possible combination of $\mathcal{X}_{hit}$.

- **Generate all possible combinations:**

```
# generate all left/right combinations (2^N x N)
choices = np.array(list(itertools.product([0, 1], repeat=N)))

# create candidate x positions for each combination ### shape (2^N, N)
x_matrix = np.where(choices == 0, x_left, x_right)

z_matrix = np.tile(z_vals, (2**N, 1)) # repeat y_vals for each combination
```

- **Implementation of least-squares fit:**

```
# linear regression by hand: x = m*y + q
z_mean = z_matrix.mean(axis=1)
x_mean = x_matrix.mean(axis=1)
m = np.sum((z_matrix-z_mean[:,None])*(x_matrix-x_mean[:,None]),axis=1)/np.sum((z_matrix-z_mean[:,None])**2,axis=1)
q = x_mean - m * z_mean
resid = np.sum((x_matrix - (m[:,None]*z_matrix + q[:,None]))**2, axis=1)
```

- **Returns the best fit:** choose the combinations of $\mathcal{X}_{hit}$ with the lowest reduced χ^2



Local fit algorithm

To apply the fitting function by partitions using `map_partitions()`, we defined the `process_partition()` function and perform the fit separately for each given Orbit and Chamber .

For each hit, we saved in a DataFrame the fit parameters, the X^2/ndof and the chosen set of best coordinates $\mathbf{x}_{\text{best}}$.

```
def process_partition(df):
    results = []
    for (orbit, chamber_id), df_grp in df.groupby(["ORBIT", "CHAMBER"]):
        best = fit_chamber_vectorized(df_grp)
        if best is not None:
            results.append({
                "ORBIT": orbit,
                "CHAMBER": chamber_id,
                "m": best["m"],
                "q": best["q"],
                "χ²/ndof": best["χ²/ndof"],
                "x_best": best["x_best"],
                "z_vals": best["z_vals"]} )

    return pd.DataFrame(results)
```

```
meta = pd.DataFrame({
    "ORBIT": pd.Series(dtype="int64"),
    "CHAMBER": pd.Series(dtype="int64"),
    "m": pd.Series(dtype="float64"),
    "q": pd.Series(dtype="float64"),
    "χ²/ndof": pd.Series(dtype="float64"),
    "x_best": pd.Series(dtype="object"),
    "z_vals": pd.Series(dtype="object")
})

df_local_fit = new_df.map_partitions(process_partition, meta=meta)
```



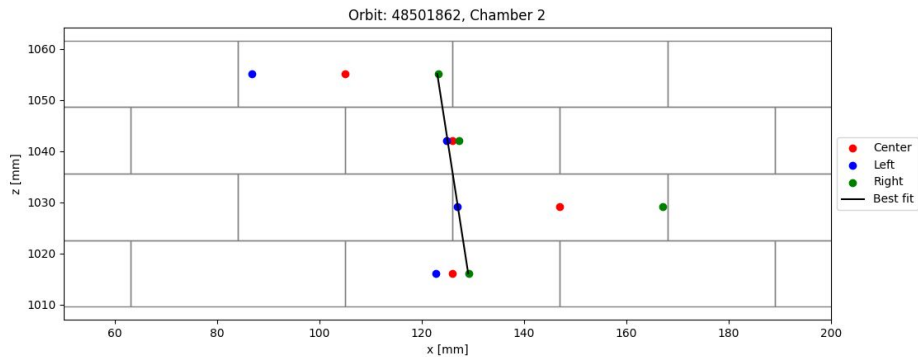
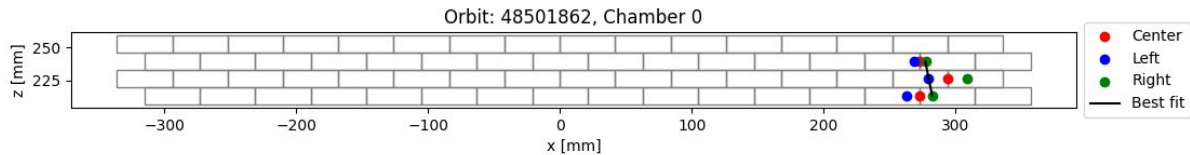
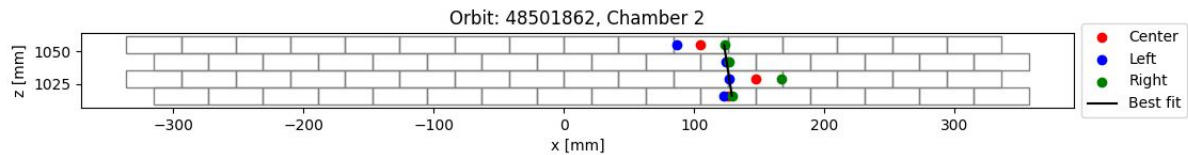
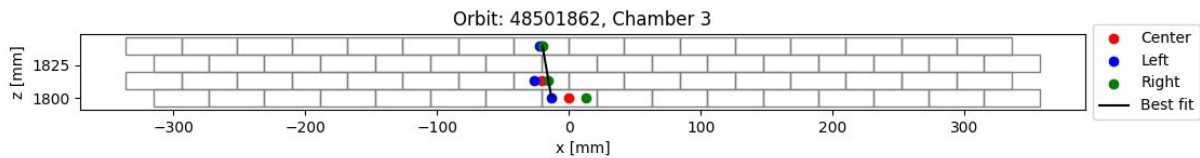
Local fit results

	ORBIT	CHAMBER	m	q	χ^2/ndof	x_best	z_vals
0	1118948	1	0.216296	-87.542832	0.775071	[119.15484644775391, 123.14315273132324, 127.8...	[957.8, 970.8, 996.8]
1	1118948	2	-0.267759	434.702037	0.690165	[162.36893305969238, 152.45173388061522, 154.8...	[1016.0999999999999, 1055.1, 1042.1, 1029.1]
2	1118948	3	-0.252889	411.517542	0.038916	[-43.832786611938474, -53.769713388061525, -50...	[1800.3, 1839.3, 1826.3, 1813.3]
3	1123056	0	0.130101	95.628744	0.667026	[124.44517950744628, 121.30684726867676, 124.2...	[226.3, 200.3, 213.3, 239.3]
4	1123056	1	0.427381	-255.825622	0.542577	[164.47251338806151, 159.94248661193848, 152.9...	[983.8, 970.8, 957.8, 996.8]
...	...	...	...	...	...	...	...
1755	48576415	1	-0.046563	135.055653	65.745969	[85.46513056640624, 91.06931474609375, 80.6499...	[983.8, 957.8, 970.8, 970.8, 983.8, 996.8]
1756	48576415	2	-0.262656	375.795920	0.086336	[105.5649, 98.8757841796875, 101.7360579101562...	[1029.1, 1055.1, 1042.1, 1016.0999999999999]
1757	48576415	3	-0.234706	339.878673	0.492665	[-82.83975361328125, -87.93993056640625, -92.3...	[1800.3, 1826.3, 1839.3, 1813.3]
1758	48580108	1	0.247827	-116.212181	13.844290	[123.14502734375, 131.8158431640625, 121.39465...	[957.8, 996.8, 970.8]
1759	48580108	2	-0.356392	468.563051	0.459022	[102.1037841796875, 92.0162841796875, 97.79071...	[1029.1, 1055.1, 1042.1, 1016.0999999999999]

18133 rows × 7 columns



Local fit plots



Global track Fit



UNIVERSITÀ
DEGLI STUDI
DI PADOVA



- We used the x_{best} points selected by the local fit to compute a **global linear fit for each Orbit**, to define the trajectory of the muons inside the whole detector.
- Only the chambers 0,2 and 3 were considered as they gather data from the same view.
- The computation were performed by partition, similarly to how they were done for the local fit.

```
def fit_global(df_orbit):  
    """  
    df_orbit: DataFrame with local fit results for chambers 0,2,3  
    and a single ORBIT.  
    It needs to contain columns 'CHAMBER', 'x_best', 'z_vals'. """  
    # Consider only chambers 0,2,3  
    df_sel = df_orbit[df_orbit["CHAMBER"].isin([0,2,3])]  
    if len(df_sel) < 2:  
        return None  
  
    # Concatenate all x_best e z_vals of  
    x_all = np.concatenate(df_sel["x_best"].values)  
    z_all = np.concatenate(df_sel["z_vals"].values)  
  
    # Global linear fit (least squares)  
    z_mean = np.mean(z_all)  
    x_mean = np.mean(x_all)  
  
    m = np.sum((z_all - z_mean)*(x_all - x_mean))/np.sum((z_all - z_mean)**2)  
    q = x_mean - m*z_mean  
  
    resid = np.sum((x_all - (m*z_all + q))**2)  
    ndof = len(x_all)-2  
    chi_sq = resid / ndof  
  
    return {  
        "m_global": m,  
        "q_global": q,  
        "chi2/ndof_global": chi_sq,  
        "x_all": x_all,  
        "z_all": z_all  
    }  
  
df_global_fit= df_local_fit.map_partitions(process_partition_global,meta=meta_global)  
global_fit_results = df_global_fit.compute()
```



Global track fit: results

	ORBIT	m_global	q_global	$\chi^2/\text{ndof_global}$	x_all	z_all
0	1118948	-0.262812	429.576638	0.260344	[162.36893305969238, 152.45173388061522, 154.8...	[1016.0999999999999, 1055.1, 1042.1, 1029.1, 1...
1	1123056	0.077788	107.109373	0.759312	[124.44517950744628, 121.30684726867676, 124.2...	[226.3, 200.3, 213.3, 239.3, 1055.1, 1029.1, 1...
2	1124284	-0.057558	148.519193	0.313131	[89.00340082092285, 90.70706694030761, 87.4432...	[1042.1, 1016.0999999999999, 1055.1, 1029.1, 1...
3	1126947	0.091026	21.586312	0.521112	[115.42823305969239, 116.2891, 118.2079, 113.5...	[1029.1, 1042.1, 1055.1, 1016.0999999999999, 1...
4	1130581	0.199468	-45.754901	102.047356	[167.7041, 151.55506694030763, 162.3241, 160.8...	[1042.1, 1029.1, 1016.0999999999999, 1055.1, 1...
...	...	...	...	...	...	...
444	48559795	0.099982	30.982830	2.039502	[50.9971568359375, 53.15100107421875, 53.91131...	[200.3, 239.3, 226.3, 213.3, 1042.1, 1016.0999...
445	48560150	0.212607	-54.569361	415.031614	[-29.1901568359375, -13.25631474609375, 18.950...	[239.3, 226.3, 200.3, 1016.0999999999999, 1029...
446	48569403	0.310180	-156.300971	0.540399	[-86.227341015625, -94.3421568359375, -90.2193...	[226.3, 200.3, 213.3, 239.3, 1042.1, 1016.0999...
447	48570850	0.230943	-64.775562	1.389592	[-12.8976568359375, -8.55068525390625, -19.129...	[226.3, 239.3, 200.3, 1016.0999999999999, 1042...
448	48576415	-0.243605	356.069091	0.255278	[105.5649, 98.8757841796875, 101.7360579101562...	[1029.1, 1055.1, 1042.1, 1016.0999999999999, 1...

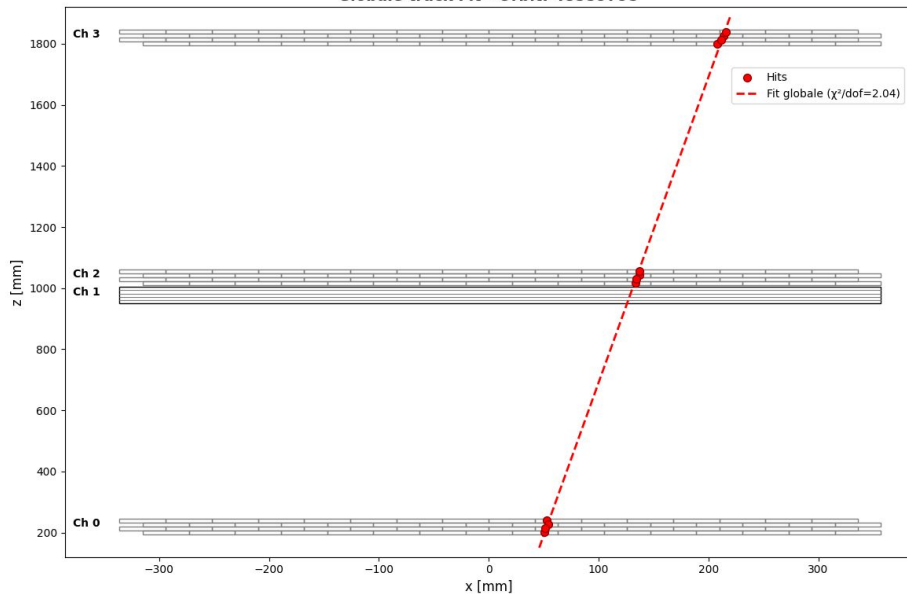
4590 rows × 6 columns



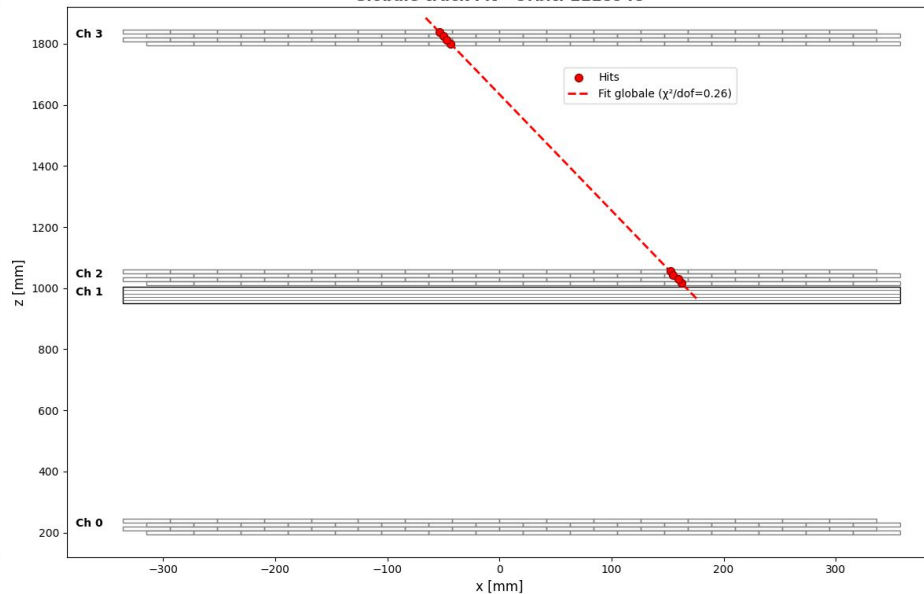
UNIVERSITÀ
DEGLI STUDI
DI PADOVA

Global fit: plots

Globale track Fit - Orbit: 48559795



Globale track Fit - Orbit: 1118948

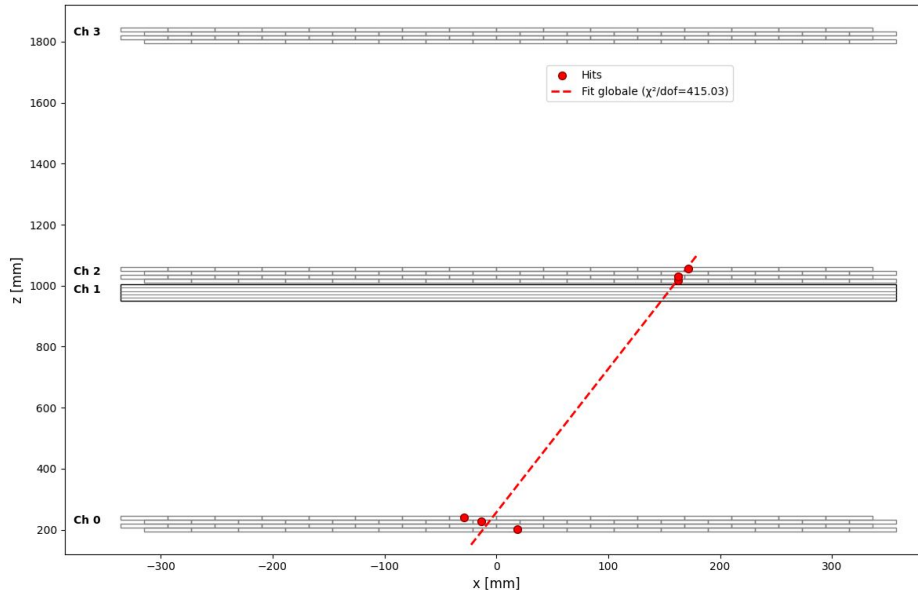




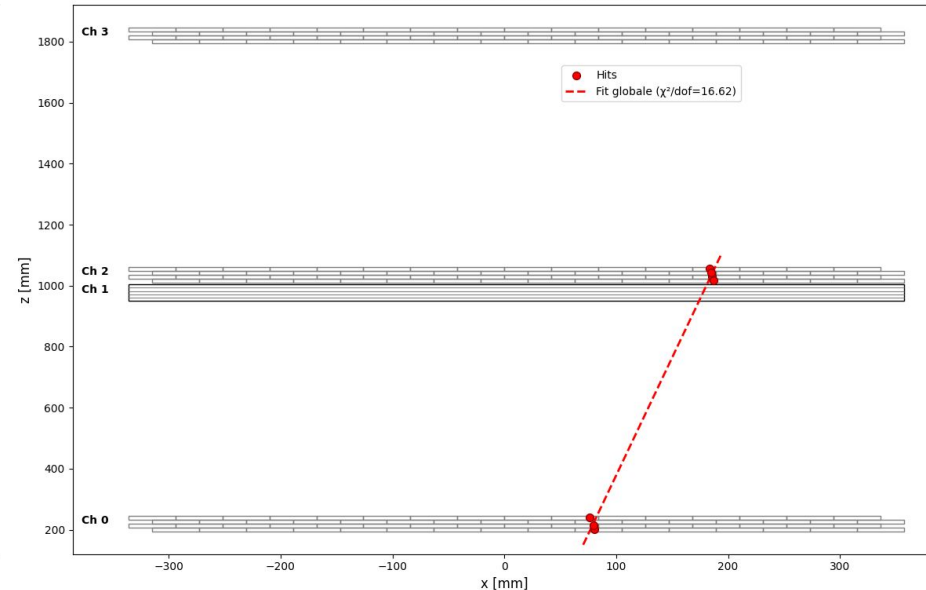
Global track fit: plots

We notice that **some events** have a very **high reduced χ^2** : these could indicate the presence of uncorrelated **background noise** or signals from **two different muons** crossing the chambers in **the same orbit** that we were unable to filter out. Some examples:

Globale track Fit - Orbit: 48560150



Globale track Fit - Orbit: 1248672

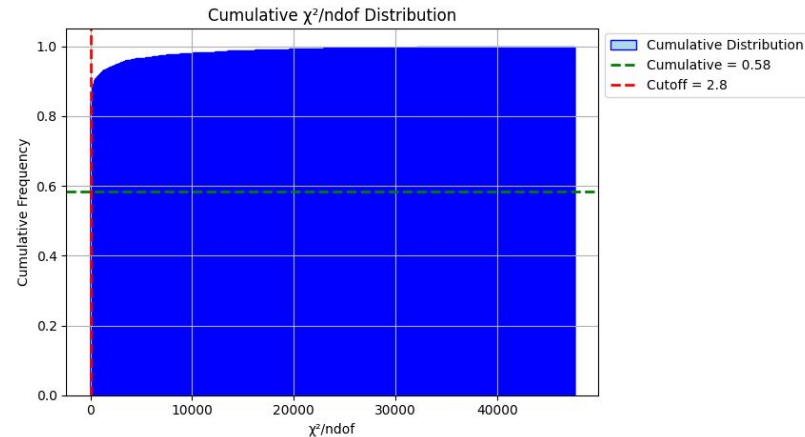
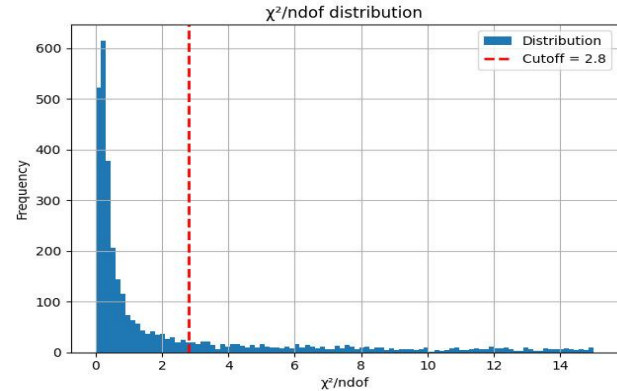


Global track fit: considerations

- Reduced χ^2 with N degrees of freedom **has** an **expected value of 1**, with $\sigma = \sqrt{2/N}$
Here we have at most $N=10 \implies \sigma \approx 0.45$
- Since we have around 4600 reconstructed events (orbits), **we should not observe fits with reduced χ^2 more than $4\sigma \approx 1.8$ away from expected value ($\chi^2 > 2.8$).**
- Instead **we observe $\approx 42\%$ of such events!**



Obviously not random fluctuations but errors



Global track fit: considerations

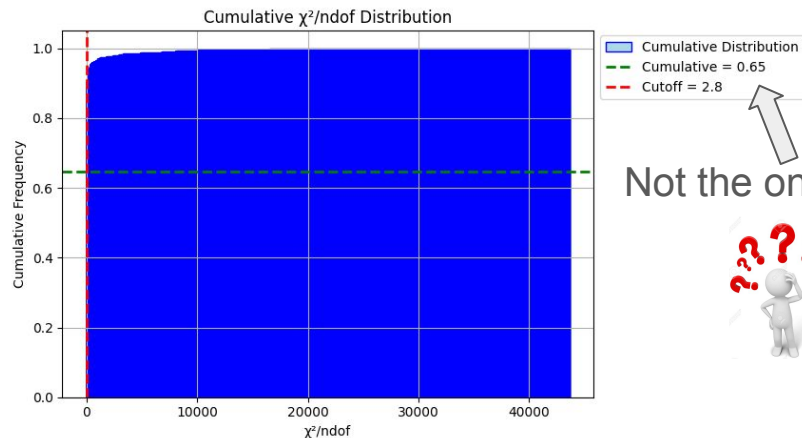
- Since we have 16 layers of cells in the detector, **we would not expect more than 16 chamber hits per orbit** if a single muon crossed the detector in that orbit (maybe scintillators might miss some muons)
- We compute how many orbits with **$num_hits > 16$** are also orbits with reduced $\chi^2 > 2.8$ in the global fit: 410/456! Such events are definitely to be discarded
- However this is definitely not the only explanation: **still a 35% of unexpected events!** Random background?

```
## Possible explanation for high chi_squared events
chamber_counts = new_df.groupby("ORBIT").size().to_frame("hits_count").reset_index()
# get orbits with more than 16 hits
possible_bad_orbits = chamber_counts[chamber_counts["hits_count"] > 16]["ORBIT"].compute()
# filter original df for those orbits
possible_bad_df = new_df[new_df["ORBIT"].isin(possible_bad_orbits.tolist())]
possible_bad_df = possible_bad_df.compute()

possible_bad_orbits = np.unique(possible_bad_df["ORBIT"])
print("Orbits with more than 16 hits:", len(possible_bad_orbits))
counts = np.sum(np.isin(bad_chi_sq_orbits, possible_bad_orbits))
print("Orbits with higher Chi squared than expected and more than 16 hits:", counts)
```

Orbits with more than 16 hits: 453

Orbits with higher Chi squared than expected and more than 16 hits: 410



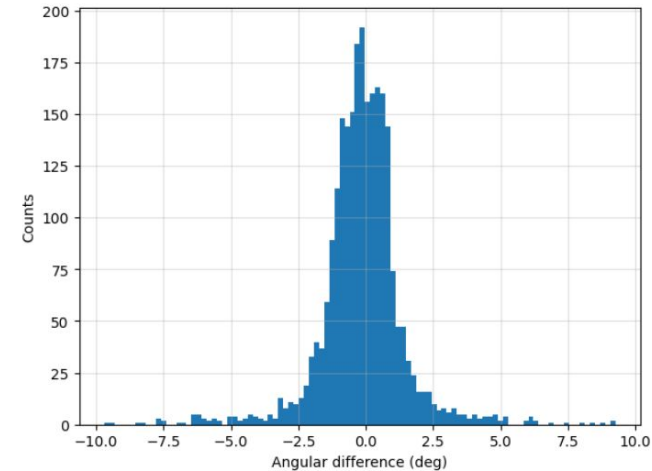
Not the only cause!



Single detector angular resolution

- We can use the global and local fit to give an estimate of the angular resolution of a single detector.
- We computed the difference between the slopes of the global track fit m_{glob} and the local track fit m_{loc} for Chamber 2, considering only events with an expected X^2 value.
- The **mean of the absolute value of the angular differences** across all events gives an estimate of the detector angular resolution. To estimate the uncertainty of the result we calculated its standard deviation.
- We obtained:

$$\theta_{res} = \frac{\sum |arctg(m_{loc}) - arctg(m_{glob})|}{N} = 1.04^\circ \pm 0.03^\circ$$



Benchmarking



UNIVERSITÀ
DEGLI STUDI
DI PADOVA



Objectives and Metrics

We evaluate the **performance** of our pipeline by testing different configurations of the Dask cluster varying:

- total number of **workers**
- **threads** per worker
- number of **partitions** in the dataset.

For each configuration, we measured the **execution time** required to **persist** the data in memory after each task in the pipeline and for the whole pipeline itself.

Additionally, we tracked the peak **memory** usage (in GiB) during the execution of the entire pipeline.



Setup and Configuration Space

We performed a full grid search over the possible configurations given by:

```
workers = [1, 2, 3]  
threads = [1, 2, 4, 8, 16]  
partitions = [2, 4, 8, 16, 32, 64, 128, 256, 512, 1024]
```

For each configuration:

- A **benchmarking run** was executed
- **Results** were **saved** in a dedicated JSON file
- Previously **completed configurations** were automatically **skipped**
- **Exceptions** during execution were caught and logged, allowing the process to continue

This approach ensured resilience against crashes and preserved partial results in case of failure.

Benchmarking

Results



UNIVERSITÀ
DEGLI STUDI
DI PADOVA



Pipeline Structure and Results Breakdown

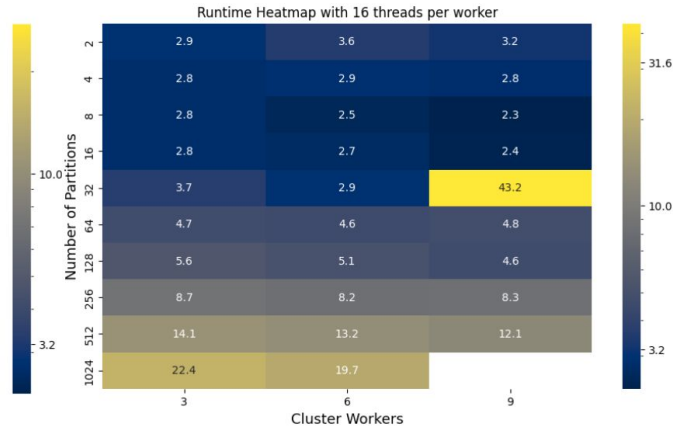
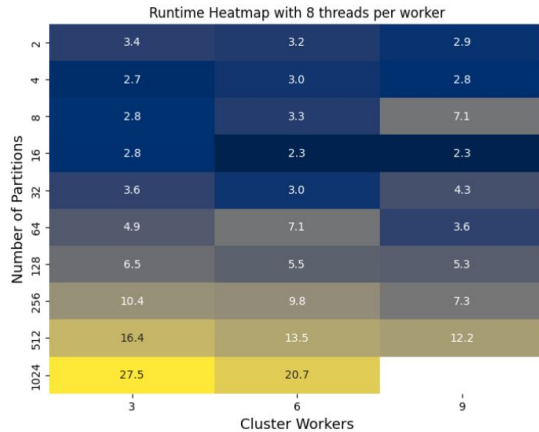
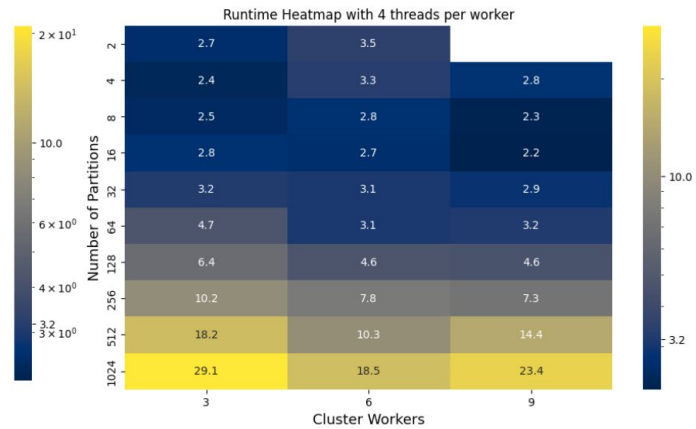
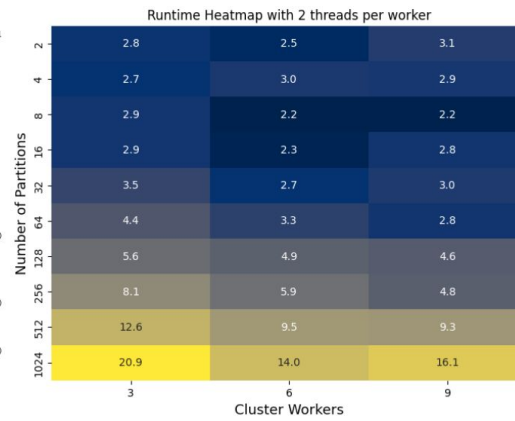
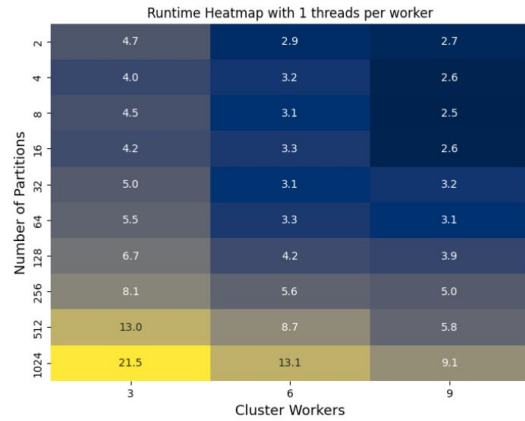
The pipeline is structured into several **sections**, each corresponding to a distinct phase of the computation:

1. Pre-processing of the dataset
2. Local track fitting
3. Global track fitting
4. Angular resolution estimation
5. Total wall time measurement
6. Peak memory usage tracking

In the following slides, we present and analyze the benchmarking results separately for each section of the pipeline, as well as for the overall execution time and memory consumption.

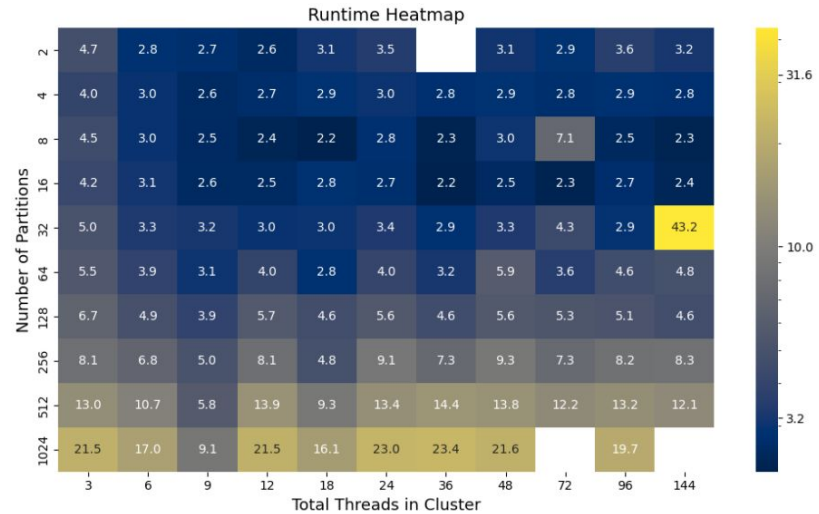
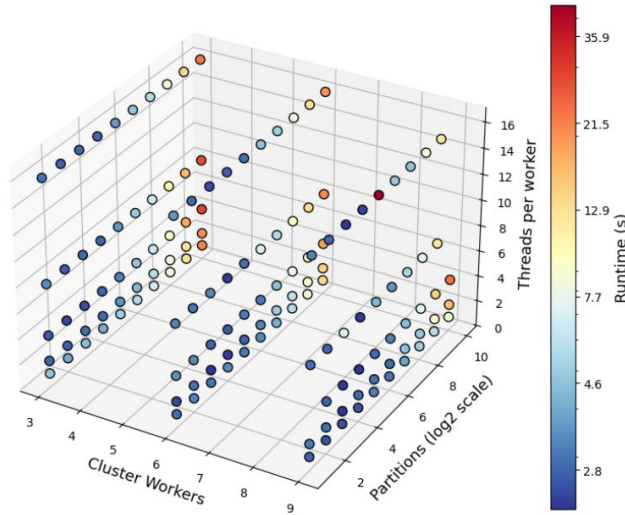


Pre-processing of the dataset



Pre-processing of the dataset

- Runtime significantly increases using many partitions
- Improvements when the total number of workers > 3
- No clear correlation between the number of threads and runtime

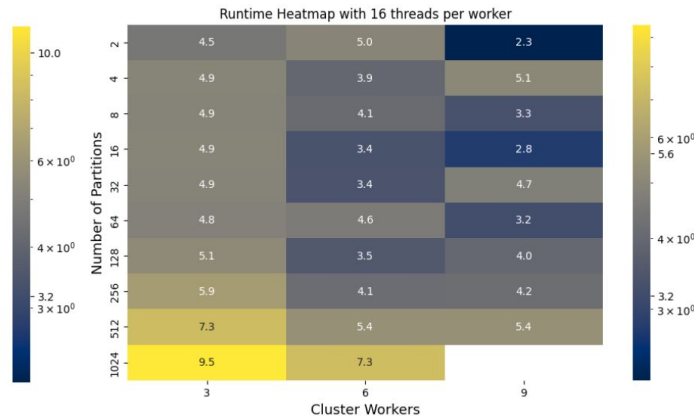
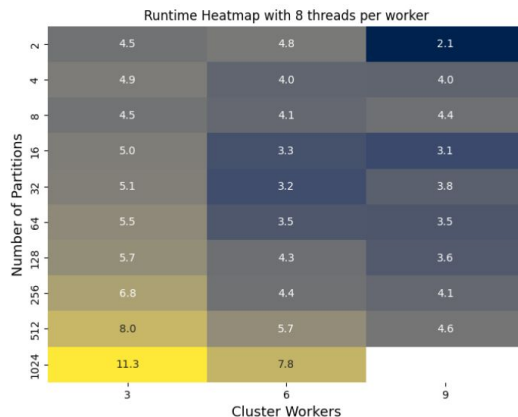
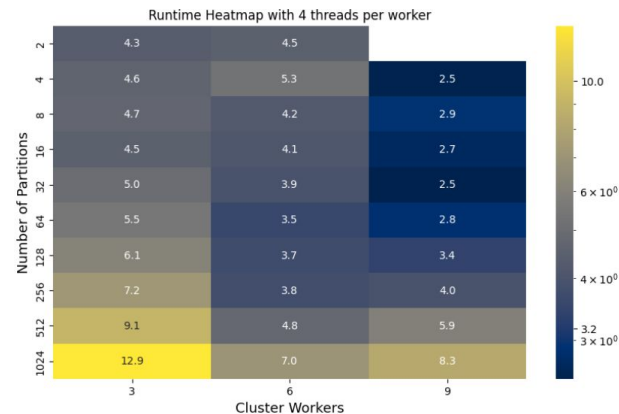
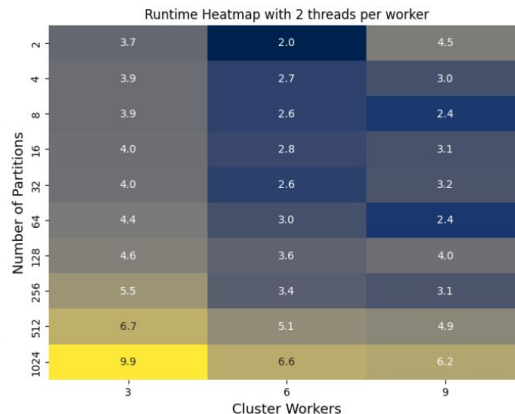
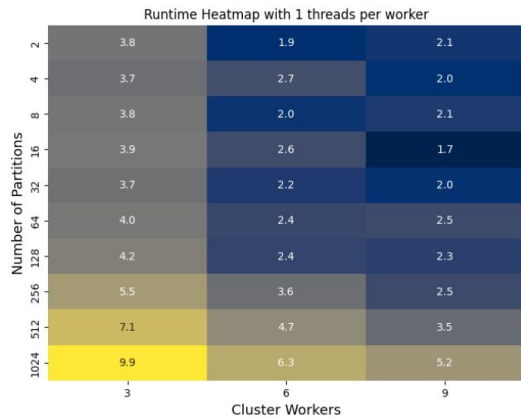


- The best results are obtained with a total number of threads > 9 and 8-16 partitions



UNIVERSITÀ
DEGLI STUDI
DI PADOVA

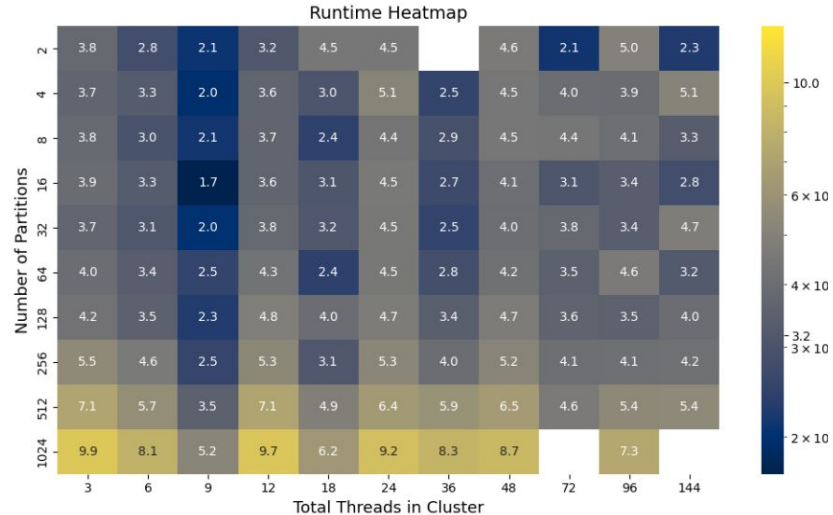
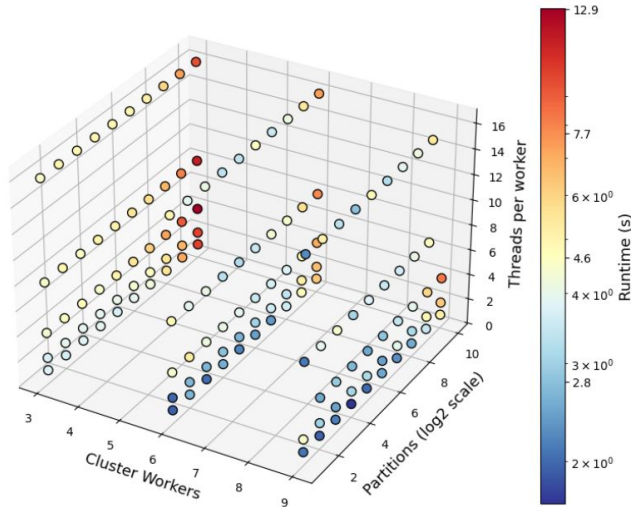
Local track Fit





Local track Fit

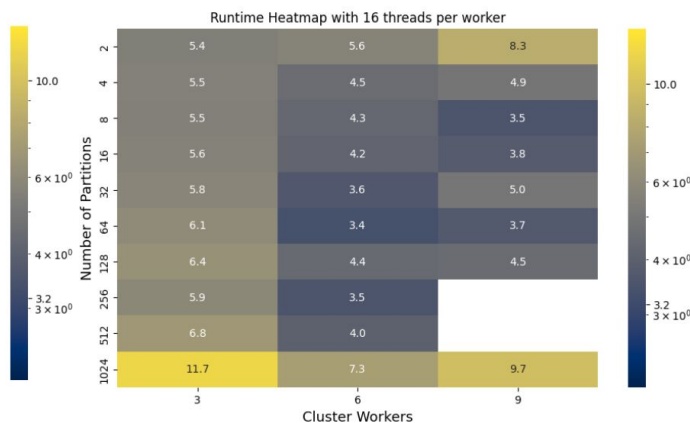
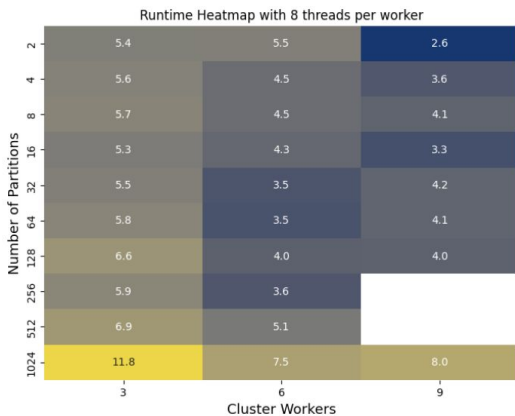
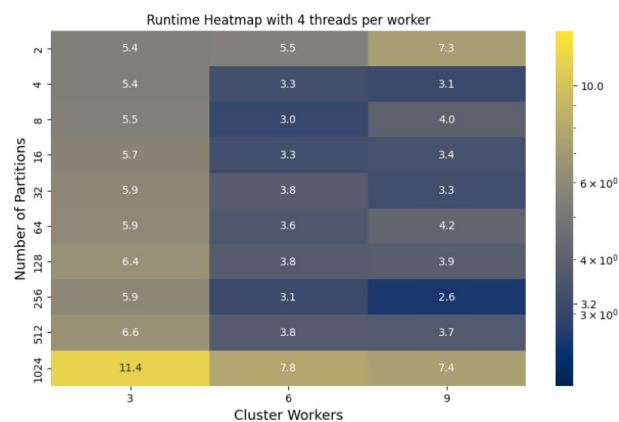
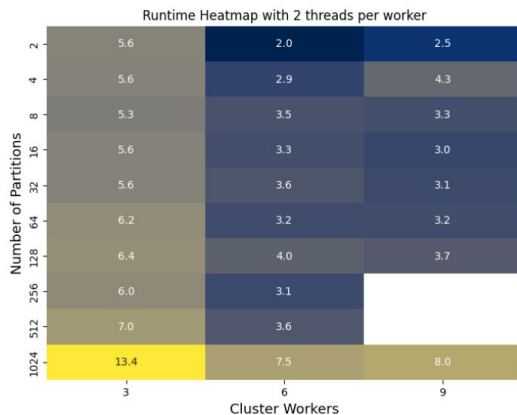
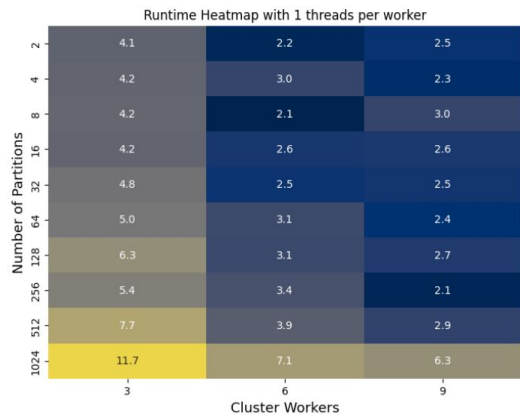
- Better computation speed with more workers in the cluster
- Performances degrade as the number of threads per worker increases
- Best setup: 6–9 workers, 1 thread per worker and partitions <32





UNIVERSITÀ
DEGLI STUDI
DI PADOVA

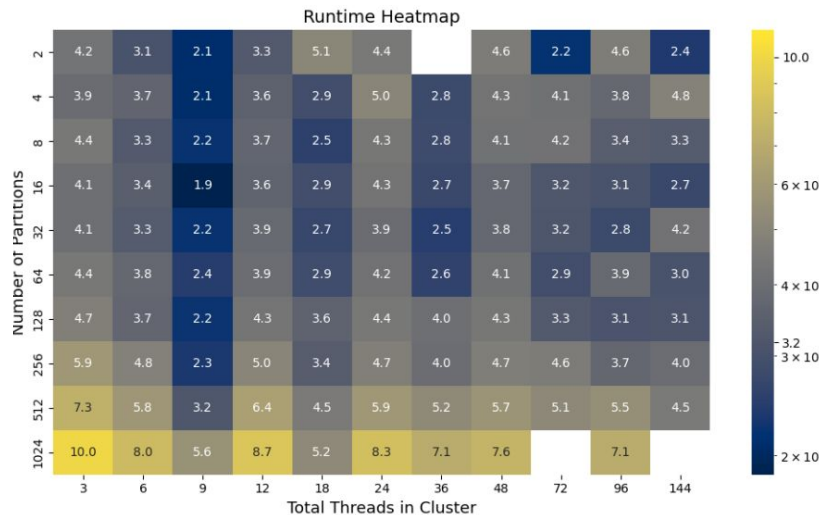
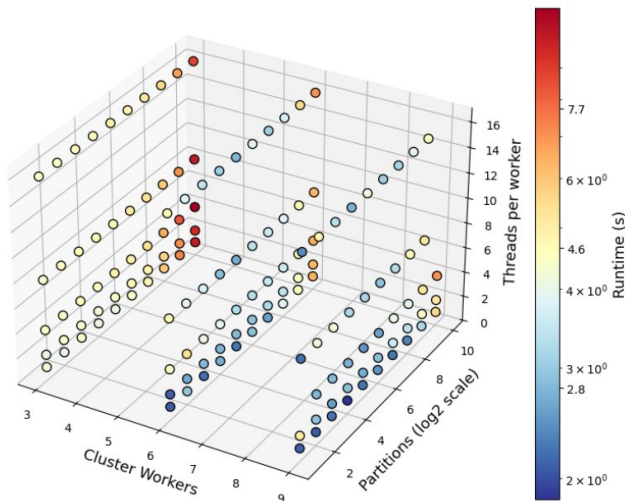
Global track Fit





Global track Fit

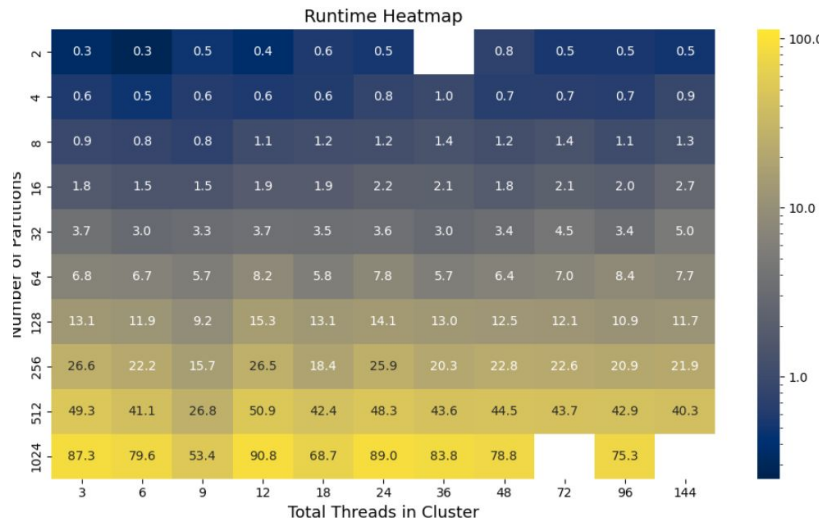
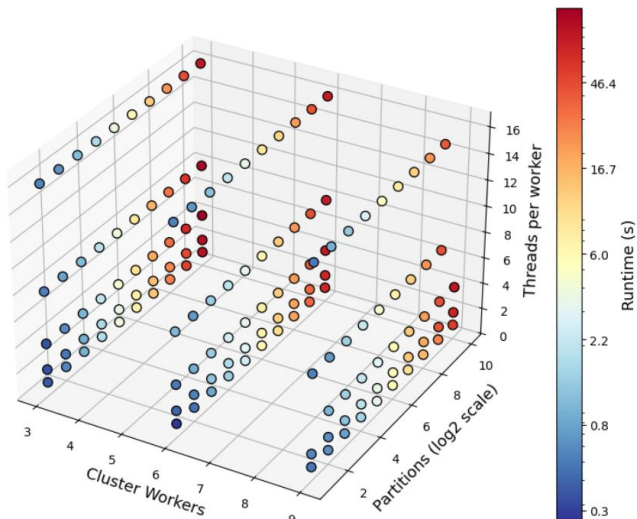
We get very similar results and analogue considerations as for the local fit.





Angular Resolution Estimation

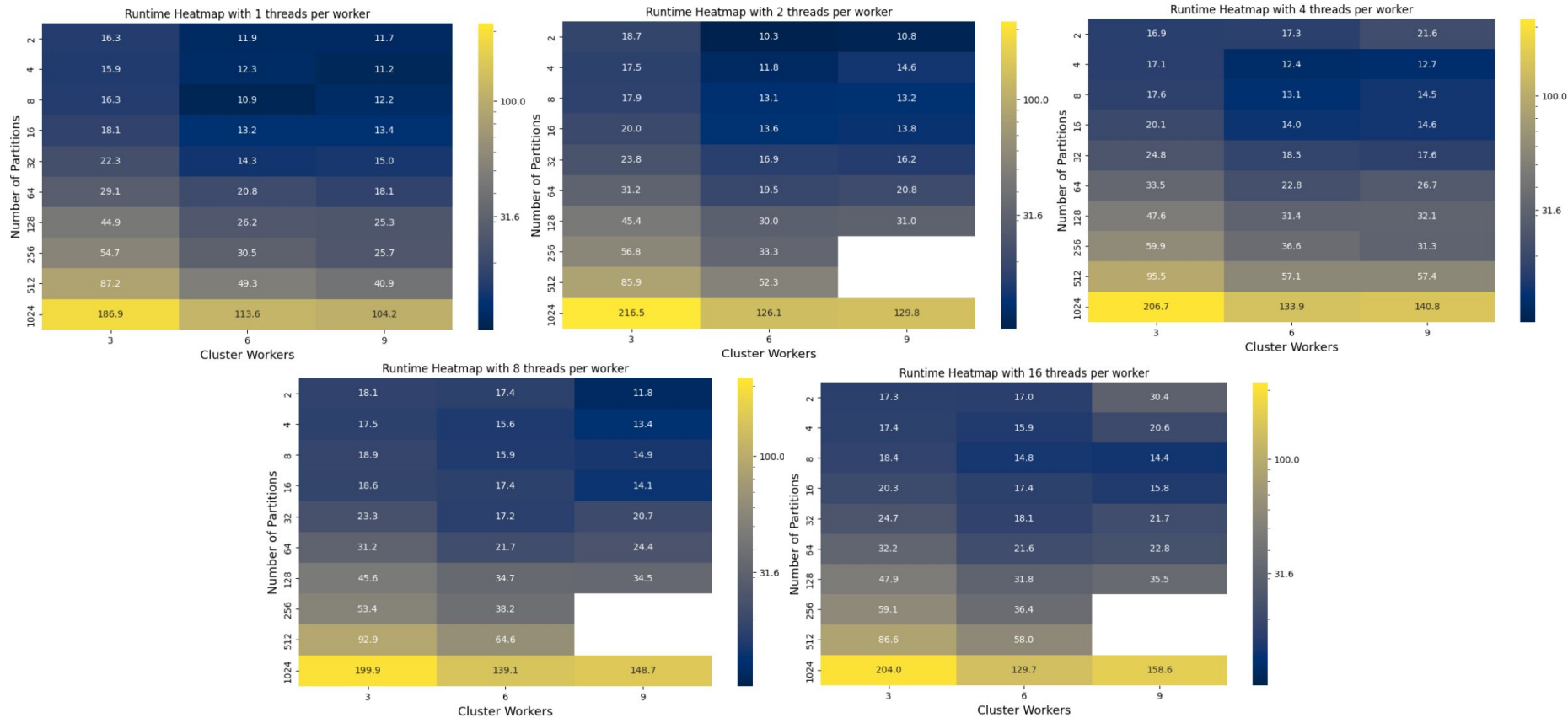
- Best results with local computation (fewer workers and partitions)
- Runtime increases with higher number of partitions





UNIVERSITÀ
DEGLI STUDI
DI PADOVA

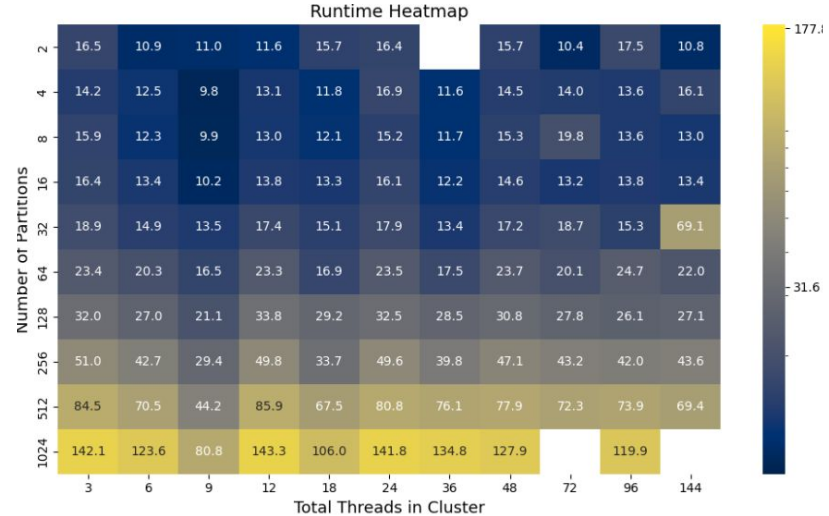
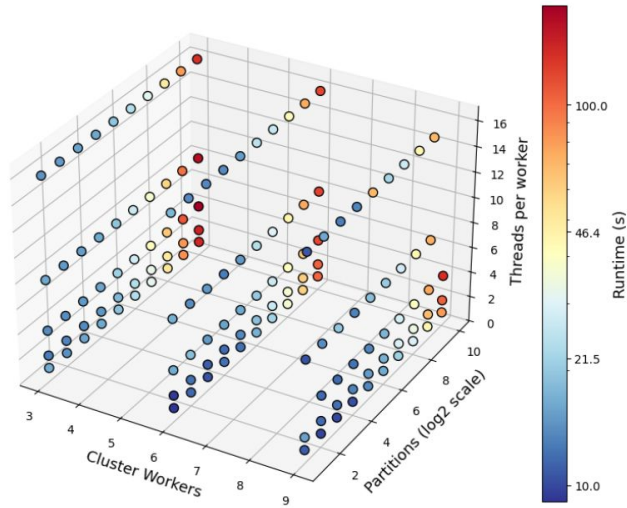
Total Wall Time





Total Wall Time

- Best compromise with 6–9 workers, 1–2 threads per worker and 4–16 partitions
- Good performance across a wide range of total threads if number of partitions < 16
- Number of partitions is the most influential parameter

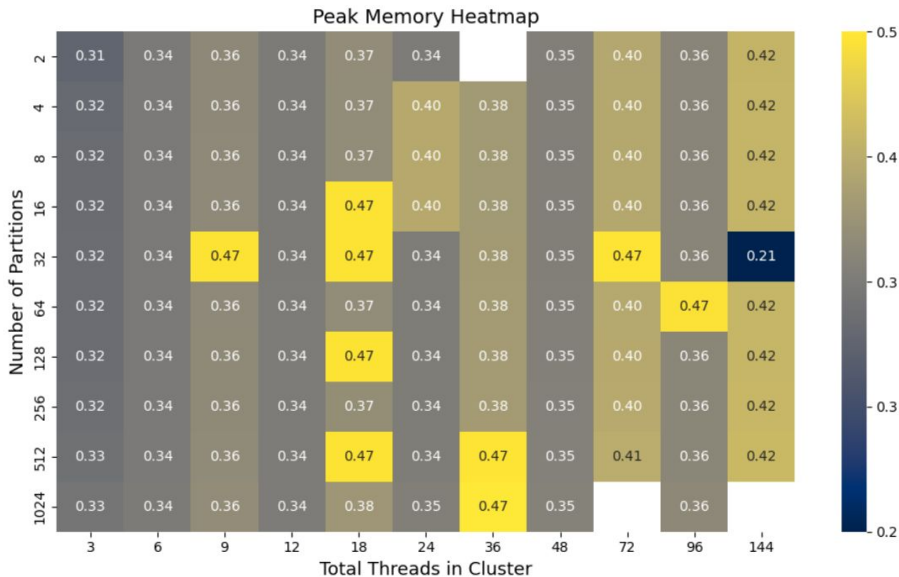




Peak Process Memory

The memory values shown here were obtained using `psutil.Process()` on the **driver**. ⚠

- This means they reflect only the driver's memory usage, **not** the workers'.
- As a result, the heatmap appears uniform.
- A correct measurement should query worker memory directly via **Dask APIs** (`client.run`).

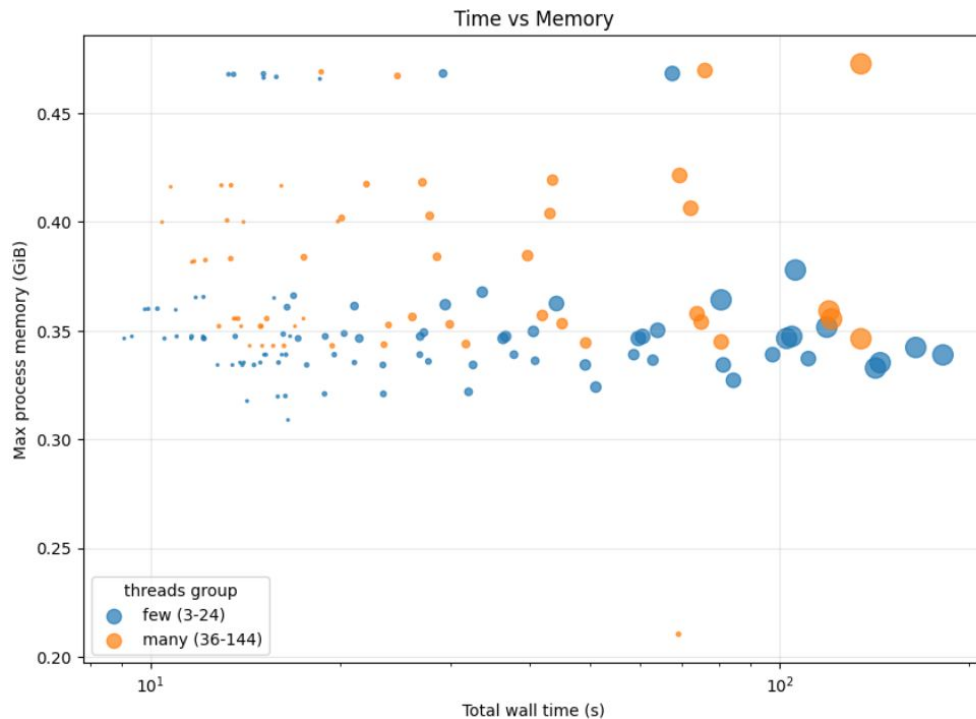




Peak Process Memory

Here the marker size represents the number of partitions used in the computation.

- **High threads** counts (●) → peak memory ~ 0.40 GiB
- **Low threads** counts (●) → peak memory ~ 0.35 GiB
- Memory usage is **stable** overall, with fluctuations within a narrow band (0.30 - 0.45 GiB).
- Too many small partitions (●) → longer execution time, i.e. increase overhead ⇒ less efficient parallelization



THANK YOU



UNIVERSITÀ
DEGLI STUDI
DI PADOVA



Peak Process Memory

Results using `MemorySampler()` from `distributed.diagnostics` to retrieve:

- **max process memory:** measured by the OS on the worker process (managed + unmanaged)
- **max managed memory:** memory temporarily allocated by the dask tasks that are currently running

with the parameters combination given

by: `workers = [1, 2, 3]`
`threads = [1, 2]`
`partitions = [16, 32]`

